

VEDAS COLLEGE

AFFILIATED TO TRIBHUVAN UNIVERSITY

JAWALAKHEL, LALITPUR



A

Lab Report

On

Design and Analysis of Algorithm

Submitted By:

Name: Prajwal Chaulagain

Symbol No: 80010773

Semester: Fifth

Faculty: Science and Technology

Level: Bachelor

Program: Computer Science and Information Technology.

Submitted To:

Instructor: Deepak Bhatta

Table of Contents

Task No.	Lab Title	Signature
Lab 1	Iterative Algorithms	
1.	GCD	
2.	Fibonacci Series	
3.	Bubble Sort	
4.	Insertion Sort	
5.	Selection Sort	
Lab 2	Divide and Conquer Algorithms	
1.	Binary Search	
2.	Min Max Finding	
3.	Merge Sort	
4.	Quick Sort	
5.	Randomized Quick Sort	
6.	Heap Sort	
Lab 3	Greedy Algorithms	
1.	Fractional Knapsack	
2.	Kruskal's Algorithm	
3.	Prim's Algorithm	
4.	Dijkstra Algorithm	
5.	Huffman Coding	
Lab 4	Dynamic Programming	
1.	Matrix Chain Multiplication	
2.	Zero One Knapsack	
3.	Floyd Warshall	
Lab 5	Backtracking	
1.	N-Queen Problem	
2.	Subset Construction Problem	

Lab 1: Iterative Algorithms

Q.N.1 GCD (Greatest Common Divisor)

Theory:

The Greatest Common Divisor (GCD) of two integers is the largest number that divides both numbers exactly. The iterative approach repeatedly applies the Euclidean algorithm until the remainder becomes zero.

Time Complexity: $O(\log(\min(a,b)))$

Space Complexity: $O(1)$

Algorithm:

- a) Start
- b) Input two integers a and b
- c) While $b \neq 0$
- d) $\text{temp} = b$
- e) $b = a \% b$
- f) $a = \text{temp}$
- g) Print a as GCD
- h) Stop

Code:

// Write a c program to find gcd of two numbers.

```
#include <stdio.h>
```

```
#include <time.h>
```

```
int main()
```

```
{
```

```
    int a, b, temp;
```

```
    clock_t start, end;
```

```
    double cpu_time_used;
```

```
    printf("Enter two integers: ");
```

```
    scanf("%d %d", &a, &b);
```

```
    start = clock();
```

```
    while (b != 0)
```

```
    {
```

```
        temp = b;
```

```
        b = a % b;
```

```
        a = temp;
```

```
    }
```

```
    end = clock();
```

```
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
```

```
    printf("GCD is %d\n", a);
```

```
    printf("Time taken: %f seconds\n", cpu_time_used);
```

```
    return 0;
```

```
}
```

Output:

```
C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 1\" && gcc 1.c -o 1 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 1\"1
Enter two integers: 5 10
GCD is 5
Time taken: 0.000000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 1>
```

Q.N.2 Fibonacci number

Theory:

The Fibonacci sequence generates numbers where each number is the sum of the previous two numbers. The iterative approach calculates values sequentially using loops instead of recursion.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Algorithm:

- a) Start
- b) Input n
- c) Initialize $a=0$, $b=1$
- d) Print a and b
- e) For $i = 2$ to n
- f) $c = a + b$
- g) Print c
- h) Update $a=b$, $b=c$
- i) Stop

Code:

// Write a c program to find nth fibonacci number.

```
#include <stdio.h>
```

```
#include <time.h>
```

```
int main()
```

```
{
```

```
    int a = 0, b = 1, n, next;
```

```
    clock_t start, end;
```

```
    double cpu_time_used;
```

```
    printf("Enter the value of n: ");
```

```
    scanf("%d", &n);
```

```
    start = clock();
```

```
    while (n - 1)
```

```
    {
```

```

        next = a + b;
        a = b;
        b = next;
        n--;
    }
    end = clock();
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Nth Fibonacci number is %d\n", a);
    printf("Time taken: %f seconds\n", cpu_time_used);
    return 0;
}

```

Output:

```

C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 1\" && gcc 2.c -o 2 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 1\"2
Enter the value of n: 5
Nth Fibonacci number is 3
Time taken: 0.000000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 1>

```

Q.N.3 Bubble Sort

Theory:

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. Larger elements move toward the end in each pass.

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Algorithm:

- a) Start
- b) Input n elements
- c) For i = 0 to n-1
- d) For j = 0 to n-i-1
- e) If $a[j] > a[j+1]$ swap
- f) Print sorted array
- g) Stop

Code:

// Write a c program to implement for Bubble Sort.

```
#include <stdio.h>
```

```
#include <time.h>
```

```
int main()
```

```
{
```

```

int arr[100], i, j, temp, n;
clock_t start, end;
double cpu_time_used;
printf("Enter number of elements (max 100): ");
scanf("%d", &n);
printf("Enter %d integers: \n", n);
for (i = 0; i < n; i++)
{
    scanf("%d", &arr[i]);
}
start = clock();
for (i = 0; i < n - 1; i++)
{
    for (j = 0; j < n - i - 1; j++)
    {
        if (arr[j] > arr[j + 1])
        {
            // swap arr[j] and arr[j+1]
            temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
end = clock();
cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("Sorted array: \n");
for (i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}
printf("\nTime taken: %f seconds\n", cpu_time_used);
return 0;
}

```

Output:

```

C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 1\" && gcc 3.c -o 3 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 1"3
Enter number of elements (max 100): 10
Enter 10 integers:
8 4 12 89 45 22 16 41 54 66
Sorted array:
4 8 12 16 22 41 45 54 66 89
Time taken: 0.000000 seconds
c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 1>

```

Q.N.4 Insertion Sort

Theory:

Insertion sort builds a sorted list by inserting each element into its correct position within the already sorted part. It is efficient for small datasets.

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Algorithm:

- a) Start
- b) Input array
- c) For $i = 1$ to $n-1$
- d) $key = a[i]$
- e) Compare with previous elements
- f) Shift larger elements
- g) Insert key in correct position
- h) Print sorted array
- i) Stop

Code:

// Write a c program to implement for Insertion Sort.

```
#include<stdio.h>
#include<time.h>

int main()
{
    int arr[100], i, j, key, n;
    clock_t start, end;
    double cpu_time_used;
    printf("Enter number of elements (max 100): ");
    scanf("%d", &n);
    printf("Enter %d integers: \n", n);
    for (i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    start = clock();
    for (i = 1; i < n; i++)
    {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
    }
    end = clock();
    cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
    printf("Time taken: %f\n", cpu_time_used);
}
```

```

    }
    arr[j + 1] = key;
}
end = clock();
cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("Sorted array: \n");
for (i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}
printf("\nTime taken: %f seconds\n", cpu_time_used);
return 0;
}

```

Output:

```

C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 1\" && gcc 4.c -o 4 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 1\"4
Enter number of elements (max 100): 5
Enter 5 integers:
45 12 27 33 10
Sorted array:
10 12 27 33 45
Time taken: 0.000000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 1>

```

Q.N.5 Selection Sort

Theory:

Selection Sort repeatedly selects the smallest element from the unsorted portion and places it at the beginning. The sorted portion grows one element at a time.

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Algorithm:

- a) Start
- b) Input array
- c) For $i = 0$ to $n-1$
- d) Find smallest element in remaining array
- e) Swap with $a[i]$
- f) Print sorted array
- g) Stop

Code:

// Write a c program to implement for Selection Sort.

```
#include <stdio.h>
```

```
#include <time.h>
```

```
int main()
```

```
{
```

```
    int arr[100], i, j, min_idx, temp, n;
```

```
    clock_t start, end;
```

```
    double cpu_time_used;
```

```
    printf("Enter number of elements (max 100): ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter %d integers: \n", n);
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    start = clock();
```

```
    for (i = 0; i < n - 1; i++)
```

```
    {
```

```
        min_idx = i;
```

```
        for (j = i + 1; j < n; j++)
```

```
        {
```

```
            if (arr[j] < arr[min_idx])
```

```
            {
```

```
                min_idx = j;
```

```
            }
```

```
        }
```

```
        // swap arr[i] and arr[min_idx]
```

```
        temp = arr[i];
```

```
        arr[i] = arr[min_idx];
```

```
        arr[min_idx] = temp;
```

```
    }
```

```
    end = clock();
```

```
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
```

```
    printf("Sorted array: \n");
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        printf("%d ", arr[i]);
```

```
    }
```

```
    printf("\nTime taken: %f seconds\n", cpu_time_used);
```

```
    return 0;
```

```
}
```

Output:

```
C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 1\" && gcc 5.c -o 5 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 1\"5
Enter number of elements (max 100): 6
Enter 6 integers:
12 45 23 77 43 45
Sorted array:
12 23 43 45 45 77
Time taken: 0.000000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 1>
```

Conclusion:

In this lab, we implemented several basic algorithms using the iterative method in C programming. The algorithms included GCD, Fibonacci sequence, Bubble Sort, Insertion Sort, and Selection Sort. We also measured the execution time of each program and analyzed their time and space complexity using Big-O notation. This lab helped us understand how iterative algorithms work and how to evaluate their efficiency.

Lab 2: Divide and Conquer Algorithms

Q.N.1 Binary Search

Theory:

Binary search is a divide and conquer algorithm used to search an element in a sorted array by repeatedly dividing the search space into halves. It compares the middle element with the target and reduces the search range accordingly.

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Algorithm:

- a) Start
- b) Input number of elements and sorted array
- c) Set low = 0 and high = n-1
- d) While low $\leq$ high
- e) Find mid = (low + high) / 2
- f) If a[mid] == key, element found
- g) If key < a[mid], set high = mid - 1
- h) Else set low = mid + 1
- i) If not found, print not found
- j) Stop

Code:

// Write a c program to implement for Binary Search.

```
#include <stdio.h>
#include <time.h>

int binarySearch(int arr[], int low, int high, int key)
{
    if (low > high)
        return -1;
    int mid = low + (high - low) / 2;
    if (arr[mid] == key)
        return mid;
    else if (arr[mid] < key)
        return binarySearch(arr, mid + 1, high, key);
    else
        return binarySearch(arr, low, mid - 1, key);
}

int main()
{
    int arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    int size = sizeof(arr) / sizeof(arr[0]);
    int key;
    clock_t start, end;
    printf("Enter element to search: ");
    scanf("%d", &key);
    start = clock();
    int result = binarySearch(arr, 0, size - 1, key);
    end = clock();
    double cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    if (result != -1)
        printf("Element found at index %d\n", result);
    else
        printf("Element not found\n");
    printf("Time taken: %f seconds\n", cpu_time_used);
    return 0;
}
```

Output:

```
C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2\" && gcc 1.c -o 1 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2\"1
Enter element to search: 12
Element found at index 3
Time taken: 0.000000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2>
```

Q.N.2 Min Max Finding

Theory:

Min-Max finding using divide and conquer splits the array into smaller parts, finds the minimum and maximum in each part, and then combines the results. This reduces the number of comparisons compared to a simple linear approach.

Time Complexity: $O(n)$

Space Complexity: $O(\log n)$

Algorithm:

- a) Start
- b) Divide array into two halves
- c) Recursively find min and max in each half
- d) Compare results of both halves
- e) Select overall minimum and maximum
- f) Stop

Code:

// Write a c program to implement for Min Max Finding.

```
#include <stdio.h>
```

```
#include <time.h>
```

```
void findMinMax(int arr[], int low, int high, int *min, int *max)
{
    if (low == high)
    {
        *min = arr[low];
        *max = arr[low];
        return;
    }
    if (high == low + 1)
    {
        if (arr[low] < arr[high])
        {
            *min = arr[low];
            *max = arr[high];
        }
        else
        {
            *min = arr[high];
            *max = arr[low];
        }
        return;
    }
    int mid = (low + high) / 2;
    int minLeft, maxLeft, minRight, maxRight;
```

```

    findMinMax(arr, low, mid, &minLeft, &maxLeft);
    findMinMax(arr, mid + 1, high, &minRight, &maxRight);
    *min = (minLeft < minRight) ? minLeft : minRight;
    *max = (maxLeft > maxRight) ? maxLeft : maxRight;
}
int main()
{
    int arr[100], n;
    clock_t start, end;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    int min, max;
    start = clock();
    findMinMax(arr, 0, n - 1, &min, &max);
    end = clock();
    double cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Minimum element = %d\n", min);
    printf("Maximum element = %d\n", max);
    printf("Time taken: %f seconds\n", cpu_time_used);
    return 0;
}

```

Output:

```

C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2\" && gcc 2.c -o 2 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2\"2
Enter number of elements: 6
Enter 6 elements:
34 67 12 44 10 61
Minimum element = 10
Maximum element = 67
Time taken: 0.000000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2>

```

Q.N.3 Merge Sort

Theory:

Merge sort is a divide and conquer sorting algorithm that divides the array into two halves, sorts each half recursively, and merges the sorted halves. It guarantees stable sorting and consistent performance.

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

Algorithm:

- a) Divide array into two halves
- b) Recursively sort each half
- c) Merge both sorted halves
- d) Repeat until array is sorted

Code:

// Write a c program to implement for Merge Sort.

```
#include <stdio.h>
```

```
#include <time.h>
```

```
void merge(int arr[], int left, int mid, int right)
```

```
{
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }
    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];
}
```

```
void mergeSort(int arr[], int left, int right)
```

```
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

```
int main()
{
```

```

int arr[100], n;
printf("Enter number of elements: ");
scanf("%d", &n);
printf("Enter %d elements:\n", n);
for (int i = 0; i < n; i++)
    scanf("%d", &arr[i]);
clock_t start = clock();
mergeSort(arr, 0, n - 1);
clock_t end = clock();
double cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("Sorted array:\n");
for (int i = 0; i < n; i++)
    printf("%d ", arr[i]);
printf("\nTime taken: %f seconds\n", cpu_time);
return 0;
}

```

Output:

```

C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2\" && gcc 3.c -o 3 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2\"3
Enter number of elements: 6
Enter 6 elements:
78 23 45 12 24 51
Sorted array:
12 23 24 45 51 78
Time taken: 0.000000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2>

```

Q.N.4 Quick Sort

Theory:

Quick sort is a divide and conquer algorithm that selects a pivot element and partitions the array so that smaller elements are on one side and larger elements are on the other side. It recursively sorts both partitions.

Time Complexity: $O(n \log n)$ average, $O(n^2)$ worst case

Space Complexity: $O(\log n)$

Algorithm:

- Choose a pivot element
- Partition array into two parts
- Recursively sort left and right parts
- Combine results

Code:

// Write a c program to implement for Quick Sort.

```
#include <stdio.h>
```

```
#include <time.h>
```

```

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high)
{
    int pivot = arr[high]; // Choosing last element as pivot
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        if (arr[j] <= pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1); // Sort left subarray
        quickSort(arr, pi + 1, high); // Sort right subarray
    }
}

int main()
{
    arr[100], n;
    clock_t start, end;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    start = clock();
    quickSort(arr, 0, n - 1);
    end = clock();
    double cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Sorted array:\n");
}

```

```

for (int i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}
printf("\nTime taken: %f seconds\n", cpu_time_used);
printf("\n");
return 0;
}

```

Output:

```

C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2\" && gcc 4.c -o 4 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2\"4
Enter number of elements: 7
Enter 7 elements:
34 12 56 39 51 20 71
Sorted array:
12 20 34 39 51 56 71
Time taken: 0.000000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2>

```

Q.N.5 Randomized Quick Sort

Theory:

Randomized quick sort improves the performance of quick sort by randomly selecting the pivot element. This reduces the probability of worst-case performance and improves average efficiency.

Time Complexity: $O(n \log n)$ expected

Space Complexity: $O(\log n)$

Algorithm:

- a) Start
- b) Input number of elements and array
- c) Select a random element from the array as pivot
- d) Swap the randomly selected pivot with the last element
- e) Partition the array so that elements smaller than pivot move to the left and larger elements move to the right
- f) Recursively apply randomized quick sort on the left subarray
- g) Recursively apply randomized quick sort on the right subarray
- h) Continue until the entire array becomes sorted
- i) Stop

Code:

// Write a c program to implement for Randomized Quick Sort.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

int randomizedPartition(int arr[], int low, int high)
{
    int random_index = low + (rand() % (high - low + 1));
    swap(&arr[random_index], &arr[high]);
    return partition(arr, low, high);
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = randomizedPartition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

void printArray(int arr[], int size)
{
    for (int i = 0; i < size; i++)
    {
```

```

        printf("%d ", arr[i]);
    }
    printf("\n");
}
int main()
{
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    clock_t start, end;
    printf("Original array: ");
    printArray(arr, n);
    start = clock();
    quickSort(arr, 0, n - 1);
    end = clock();
    double cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Sorted array: ");
    printArray(arr, n);
    printf("Time taken: %f seconds\n", cpu_time_used);
    return 0;
}

```

Output:

```

C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2\" && gcc 5.c -o 5 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2\"5
Original array: 10 7 8 9 1 5
Sorted array: 1 5 7 8 9 10
Time taken: 0.000000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2>

```

Q.N.6 Heap Sort

Theory:

Heap sort uses a binary heap data structure to sort elements. It first builds a max heap and repeatedly extracts the maximum element and places it at the end of the array.

Time Complexity: $O(n \log n)$

Space Complexity: $O(1)$

Algorithm:

- a) Start
- b) Input number of elements and array
- c) Build a max heap from the given array
- d) Swap the first element (largest) with the last element
- e) Reduce the heap size by one
- f) Apply heapify operation on the root to maintain heap property

- g) Repeat the process until all elements are sorted
- h) Print the sorted array
- i) Stop

Code:

// Write a c program to implement for Heap Sort.

```
#include <stdio.h>
#include <time.h>

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;
    if (largest != i)
    {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i >= 1; i--)
    {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}

void printArray(int arr[], int size)
{
    for (int i = 0; i < size; i++)
    {
        printf("%d ", arr[i]);
    }
}
```

```

    printf("\n");
}
int main()
{
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(arr) / sizeof(arr[0]);
    clock_t start, end;
    start = clock();
    printf("Original array: ");
    printArray(arr, n);
    heapSort(arr, n);
    end = clock();
    double cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Sorted array: ");
    printArray(arr, n);
    printf("Time taken: %f seconds\n", cpu_time_used);
    return 0;
}

```

Output:

```

C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2\" && gcc 6.c -o 6 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2\"6
Original array: 12 11 13 5 6 7
Sorted array: 5 6 7 11 12 13
Time taken: 0.002000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 2>

```

Conclusion:

In this lab, several algorithms based on the divide and conquer technique were implemented and analyzed using C programming. The algorithms included Binary Search, Min-Max finding, Merge Sort, Quick Sort, Randomized Quick Sort, and Heap Sort. These algorithms work by dividing the problem into smaller subproblems, solving them recursively, and then combining the results to obtain the final solution.

Lab 3: Greedy Algorithms

Q.N.1 Fractional Knapsack Problem.

Theory:

Fractional knapsack is a greedy algorithm used to maximize profit when items can be divided. Items are selected based on the highest profit-to-weight ratio first.

Time Complexity: $O(n \log n)$ due to sorting

Space Complexity: $O(1)$

Algorithm:

- a) Start
- b) Input number of items
- c) Input weight and profit of each item
- d) Calculate profit/weight ratio for each item
- e) Sort the items in decreasing order of profit/weight ratio
- f) Initialize total profit = 0
- g) For each item in sorted order
- h) If the item weight is less than or equal to remaining capacity
- i) Add the whole item to the knapsack
- j) Decrease the capacity accordingly
- k) Otherwise add the fractional part of the item
- l) Add corresponding profit to total profit
- m) Stop when knapsack becomes full
- n) Print maximum profit
- o) Stop

Code:

// Write a c program to implement for Fractional Knapsack Problem.

```
#include <stdio.h>
```

```
#include <time.h>
```

```
struct Item
```

```
{
```

```
    int value;
```

```
    int weight;
```

```
    float ratio;
```

```
};
```

```
void swap(struct Item *a, struct Item *b)
```

```
{
```

```
    struct Item temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
void sortItems(struct Item arr[], int n)
```

```

{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j].ratio < arr[j + 1].ratio)
            {
                swap(&arr[j], &arr[j + 1]);
            }
        }
    }
}

float fractionalKnapsack(int capacity, struct Item arr[], int n)
{
    sortItems(arr, n);
    float totalValue = 0.0;
    for (int i = 0; i < n; i++)
    {
        if (capacity >= arr[i].weight)
        {
            capacity -= arr[i].weight;
            totalValue += arr[i].value;
        }
        else
        {
            totalValue += arr[i].ratio * capacity;
            break;
        }
    }
    return totalValue;
}

int main()
{
    int n, capacity;
    clock_t start, end;
    double cpu_time_used;
    printf("Enter number of items: ");
    scanf("%d", &n);
    struct Item arr[n];
    printf("Enter weight and value for each item:\n");
    for (int i = 0; i < n; i++)
    {
        scanf("%d %d", &arr[i].weight, &arr[i].value);
        arr[i].ratio = (float)arr[i].value / arr[i].weight;
    }
    printf("Enter knapsack capacity: ");

```

```

scanf("%d", &capacity);
start = clock();
float maxValue = fractionalKnapsack(capacity, arr, n);
printf("Maximum value = %.2f\n", maxValue);
cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("Time taken: %f seconds\n", cpu_time_used);
return 0;
}

```

Output:

```

C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 3\" && gcc 1.c -o 1 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 3\"1
Enter number of items: 3
Enter weight and value for each item:
10 60
20 100
30 120
Enter knapsack capacity: 50
Maximum value = 240.00
Time taken: 0.000000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 3>

```

Q.N.2 Kruskal Algorithms

Theory:

Kruskal's algorithm is a greedy algorithm used to find the minimum spanning tree of a graph. It selects the smallest edge that does not form a cycle until all vertices are connected.

Time Complexity: $O(E \log E)$

Space Complexity: $O(V)$

Algorithm:

- a) Start
- b) Input number of vertices and edges
- c) Input all edges with their weights
- d) Sort all edges in increasing order of weight
- e) Initialize each vertex as a separate set
- f) Select the smallest edge from the sorted list
- g) Check whether adding the edge forms a cycle
- h) If it does not form a cycle, include the edge in the minimum spanning tree
- i) Otherwise discard the edge
- j) Repeat the process until the minimum spanning tree contains $(V-1)$ edges
- k) Print the edges of the minimum spanning tree
- l) Stop

Code:

// Write a c program to implement for Kruskal Algorithms.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

struct Edge
{
    int src, dest, weight;
};
struct Subset
{
    int parent;
    int rank;
};
int find(struct Subset subsets[], int i)
{
    if (subsets[i].parent != i)
        subsets[i].parent = find(subsets, subsets[i].parent);
    return subsets[i].parent;
}
void Union(struct Subset subsets[], int x, int y)
{
    int rootX = find(subsets, x);
    int rootY = find(subsets, y);
    if (subsets[rootX].rank < subsets[rootY].rank)
        subsets[rootX].parent = rootY;
    else if (subsets[rootX].rank > subsets[rootY].rank)
        subsets[rootY].parent = rootX;
    else
    {
        subsets[rootY].parent = rootX;
        subsets[rootX].rank++;
    }
}
int compare(const void *a, const void *b)
{
    struct Edge *e1 = (struct Edge *)a;
    struct Edge *e2 = (struct Edge *)b;
    return e1->weight - e2->weight;
}
void kruskal(struct Edge edges[], int V, int E)
{
    qsort(edges, E, sizeof(edges[0]), compare);
    struct Subset subsets[V];
    for (int v = 0; v < V; v++)
```

```

{
    subsets[v].parent = v;
    subsets[v].rank = 0;
}
struct Edge result[V];
int e = 0; // result index
int i = 0; // sorted edges index
while (e < V - 1 && i < E)
{
    struct Edge next = edges[i++];
    int x = find(subsets, next.src);
    int y = find(subsets, next.dest);
    if (x != y)
    {
        result[e++] = next;
        Union(subsets, x, y);
    }
}
printf("Edges in MST:\n");
int total = 0;
for (i = 0; i < e; i++)
{
    printf("%d -- %d == %d\n", result[i].src, result[i].dest, result[i].weight);
    total += result[i].weight;
}
printf("Total cost of MST = %d\n", total);
}

int main()
{
    int V, E;
    clock_t start, end;
    double cpu_time_used;
    printf("Enter number of vertices and edges: ");
    scanf("%d %d", &V, &E);
    struct Edge edges[E];
    printf("Enter each edge (src dest weight):\n");
    for (int i = 0; i < E; i++)
        scanf("%d %d %d", &edges[i].src, &edges[i].dest, &edges[i].weight);
    kruskal(edges, V, E);
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Time taken: %f seconds\n", cpu_time_used);
    return 0;
}

```

Output:

```
C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 3\" && gcc 2.c -o 2 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 3\"2
Enter number of vertices and edges: 4 5
Enter each edge (src dest weight):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
Edges in MST:
2 -- 3 == 4
0 -- 3 == 5
0 -- 1 == 10
Total cost of MST = 19
Time taken: 1959914.116000 seconds
```

Q.N.3 Prim's Algorithms

Theory:

Prim's algorithm is a greedy algorithm used to find the minimum spanning tree of a connected weighted graph. It starts from a vertex and repeatedly adds the smallest edge that connects a visited vertex to an unvisited vertex.

Time Complexity: $O(V^2)$

Space Complexity: $O(V)$

Algorithm:

- Start
- Input number of vertices and adjacency matrix
- Select a starting vertex and mark it as visited
- Find the smallest edge connecting a visited vertex to an unvisited vertex
- Add the edge to the minimum spanning tree
- Mark the new vertex as visited
- Repeat the process until all vertices are included
- Print the edges of the minimum spanning tree
- Stop

Code:

// Write a c program to implement for Prim's Algorithms.

```
#include <stdio.h>
#include <limits.h>
#include <time.h>
#define MAX 100
```

```
int minKey(int key[], int mstSet[], int V)
{
    int min = INT_MAX, min_index;
    for (int v = 0; v < V; v++)
```

```

        if (mstSet[v] == 0 && key[v] < min)
        {
            min = key[v];
            min_index = v;
        }
    return min_index;
}

void printMST(int parent[], int graph[MAX][MAX], int V)
{
    int total = 0;
    printf("Edge \tWeight\n");
    for (int i = 1; i < V; i++)
    {
        printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
        total += graph[i][parent[i]];
    }
    printf("Total cost = %d\n", total);
}

void primMST(int graph[MAX][MAX], int V)
{
    int parent[V];
    int key[V];
    int mstSet[V];
    for (int i = 0; i < V; i++)
    {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }
    key[0] = 0;
    parent[0] = -1;
    for (int count = 0; count < V - 1; count++)
    {
        int u = minKey(key, mstSet, V);
        mstSet[u] = 1;
        for (int v = 0; v < V; v++)
        {
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v])
            {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }
    printMST(parent, graph, V);
}

int main()

```

```

{
    int V;
    int graph[MAX][MAX];
    clock_t start, end;
    double cpu_time_used;
    printf("Enter number of vertices: ");
    scanf("%d", &V);
    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            scanf("%d", &graph[i][j]);
    primMST(graph, V);
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Time taken: %f seconds\n", cpu_time_used);
    return 0;
}

```

Output:

```

C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 3\" && gcc 3.c -o 3 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 3\"3
Enter number of vertices: 5
Enter adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0
Edge      Weight
0 - 1     2
1 - 2     3
0 - 3     6
1 - 4     5
Total cost = 16
Time taken: -4161.728000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 3>

```

Q.N.4 Dijkstra Algorithms

Theory:

Dijkstra's algorithm is a greedy algorithm used to find the shortest path from a source vertex to all other vertices in a weighted graph with non-negative edge weights.

Time Complexity: $O(V^2)$

Space Complexity: $O(V)$

Algorithm:

- a) Start
- b) Input number of vertices and cost matrix

- c) Select the source vertex
- d) Initialize distance array with infinity
- e) Set distance of source vertex to 0
- f) Mark all vertices as unvisited
- g) Select the unvisited vertex with the smallest distance
- h) Mark it as visited
- i) Update distances of its adjacent vertices
- j) Repeat until all vertices are visited
- k) Print shortest distances from the source
- l) Stop

Code:

// Write a c program to implement for Dijkstra Algorithms.

```
#include <stdio.h>
#include <limits.h>
#include <time.h>
#define MAX 100
int minDistance(int dist[], int visited[], int V)
{
    int min = INT_MAX, min_index;
    for (int i = 0; i < V; i++)
        if (!visited[i] && dist[i] <= min)
        {
            min = dist[i];
            min_index = i;
        }
    return min_index;
}
void dijkstra(int graph[MAX][MAX], int V, int src)
{
    int dist[V];
    int visited[V];
    for (int i = 0; i < V; i++)
    {
        dist[i] = INT_MAX;
        visited[i] = 0;
    }
    dist[src] = 0;
    for (int count = 0; count < V - 1; count++)
    {
        int u = minDistance(dist, visited, V);
        visited[u] = 1;
        for (int v = 0; v < V; v++)
        {
            if (!visited[v] &&
                graph[u][v] &&
```

```

        dist[u] != INT_MAX &&
        dist[u] + graph[u][v] < dist[v])
    {
        dist[v] = dist[u] + graph[u][v];
    }
}
printf("Vertex\tDistance from Source\n");
for (int i = 0; i < V; i++)
    printf("%d\t%d\n", i, dist[i]);
}
int main()
{
    int V, src;
    int graph[MAX][MAX];
    clock_t start, end;
    printf("Enter number of vertices: ");
    scanf("%d", &V);
    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            scanf("%d", &graph[i][j]);

    printf("Enter source vertex: ");
    scanf("%d", &src);
    start = clock();
    dijkstra(graph, V, src);
    end = clock();
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Execution time: %f seconds\n", time_taken);
    return 0;
}

```

Output:

```

C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 3\" && gcc 4.c -o 4 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 3\"4
Enter number of vertices: 4
Enter adjacency matrix:
0 5 0 10
5 0 15 0
0 15 0 20
10 0 20 0
Enter source vertex: 0
Vertex Distance from Source
0      0
1      5
2      20
3      10
Execution time: 0.000000 seconds
c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 3>

```

Q.N.5 Huffman Coding

Theory:

Huffman coding is a greedy algorithm used for data compression. It builds a binary tree where characters with smaller frequencies are placed deeper in the tree to minimize total encoding length.

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

Algorithm:

- a) Start
- b) Input characters and their frequencies
- c) Create a leaf node for each character
- d) Insert all nodes into a priority queue based on frequency
- e) Remove the two nodes with the smallest frequency
- f) Create a new node with these two nodes as children
- g) Assign the sum of their frequencies as the new node frequency
- h) Insert the new node back into the queue
- i) Repeat until only one node remains
- j) The remaining node becomes the root of the Huffman tree
- k) Traverse the tree to generate binary codes for each character
- l) Print Huffman codes
- m) Stop

Code:

// Write a c program to implement for Huffman Coding.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 100

// Huffman Tree Node
struct Node
{
    char data;
    unsigned freq;
    struct Node *left, *right;
};

// Min Heap structure
struct MinHeap
{
    unsigned size;
    unsigned capacity;
    struct Node *array[MAX];
};
```

```

// Create node
struct Node *newNode(char data, unsigned freq)
{
    struct Node *temp = (struct Node *)malloc(sizeof(struct Node));
    temp->left = temp->right = NULL;
    temp->data = data;
    temp->freq = freq;
    return temp;
}

// Create Min Heap
struct MinHeap *createMinHeap(unsigned capacity)
{
    struct MinHeap *minHeap = (struct MinHeap *)malloc(sizeof(struct MinHeap));
    minHeap->size = 0;
    minHeap->capacity = capacity;
    return minHeap;
}

// Swap nodes
void swap(struct Node **a, struct Node **b)
{
    struct Node *t = *a;
    *a = *b;
    *b = t;
}

// Heapify
void minHeapify(struct MinHeap *heap, int idx)
{
    int smallest = idx;
    int left = 2 * idx + 1;
    int right = 2 * idx + 2;
    if (left < heap->size && heap->array[left]->freq < heap->array[smallest]->freq)
        smallest = left;
    if (right < heap->size && heap->array[right]->freq < heap->array[smallest]->freq)
        smallest = right;
    if (smallest != idx)
    {
        swap(&heap->array[smallest], &heap->array[idx]);
        minHeapify(heap, smallest);
    }
}

// Extract minimum
struct Node *extractMin(struct MinHeap *heap)
{
    struct Node *temp = heap->array[0];
    heap->array[0] = heap->array[heap->size - 1];
    heap->size--;
}

```

```

        minHeapify(heap, 0);
        return temp;
    }
    // Insert into heap
    void insertHeap(struct MinHeap *heap, struct Node *node)
    {
        heap->size++;
        int i = heap->size - 1;
        while (i && node->freq < heap->array[(i - 1) / 2]->freq)
        {
            heap->array[i] = heap->array[(i - 1) / 2];
            i = (i - 1) / 2;
        }
        heap->array[i] = node;
    }
    // Build initial heap
    void buildHeap(struct MinHeap *heap)
    {
        int n = heap->size - 1;
        for (int i = (n - 1) / 2; i >= 0; i--)
            minHeapify(heap, i);
    }
    // Print codes
    void printCodes(struct Node *root, int arr[], int top)
    {
        if (root->left)
        {
            arr[top] = 0;
            printCodes(root->left, arr, top + 1);
        }
        if (root->right)
        {
            arr[top] = 1;
            printCodes(root->right, arr, top + 1);
        }
        if (!root->left && !root->right)
        {
            printf("%c: ", root->data);
            for (int i = 0; i < top; i++)
                printf("%d", arr[i]);
            printf("\n");
        }
    }
    // Build Huffman Tree
    struct Node *buildHuffman(char data[], int freq[], int n)
    {

```

```

struct MinHeap *heap = createMinHeap(n);
for (int i = 0; i < n; i++)
    heap->array[heap->size++] = newNode(data[i], freq[i]);

buildHeap(heap);
while (heap->size > 1)
{
    struct Node *left = extractMin(heap);
    struct Node *right = extractMin(heap);
    struct Node *top = newNode('$', left->freq + right->freq);
    top->left = left;
    top->right = right;
    insertHeap(heap, top);
}
return extractMin(heap);
}
// Driver
int main()
{
    int n;
    clock_t start, end;
    char data[MAX];
    int freq[MAX];
    printf("Enter number of characters: ");
    scanf("%d", &n);
    printf("Enter characters:\n");
    for (int i = 0; i < n; i++)
        scanf(" %c", &data[i]);
    printf("Enter frequencies:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &freq[i]);
    start = clock();
    struct Node *root = buildHuffman(data, freq, n);
    int arr[MAX], top = 0;
    printf("\nHuffman Codes:\n");
    printCodes(root, arr, top);
    end = clock();
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Execution time: %f seconds\n", time_taken);
    return 0;
}

```

Output:

```
C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 3\" && gcc 5.c -o 5 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 3\"5
Enter number of characters: 4
Enter characters:
A B C D
Enter frequencies:
5 9 12 13

Huffman Codes:
A: 00
B: 01
C: 10
D: 11
Execution time: 0.003000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 3>
```

Conclusion:

In this lab, several problems were solved using the greedy algorithm approach. The algorithms implemented included Fractional Knapsack, Kruskal algorithm, Prim algorithm, Dijkstra algorithm, and Huffman coding. These algorithms follow the greedy strategy where the best local choice is selected at each step to achieve an optimal or near optimal solution.

Lab 4: Dynamic Programming

Q.N.1 Matrix Chain Multiplication

Theory:

Matrix Chain Multiplication is a dynamic programming technique used to determine the most efficient way to multiply a sequence of matrices. It minimizes the number of scalar multiplications.

Time Complexity: $O(n^3)$

Space Complexity: $O(n^2)$

Algorithm:

- a) Start
- b) Input number of matrices
- c) Input dimensions of matrices
- d) Create a table to store minimum multiplication cost
- e) Initialize cost of multiplying one matrix as zero
- f) Compute cost for chain length from 2 to n
- g) For each possible position split the chain
- h) Calculate multiplication cost and update minimum cost
- i) Store minimum cost in the table
- j) Print minimum multiplication cost
- k) Stop

Code:

// Write a c program to implement for Matrix Chain Multiplication.

```
#include <stdio.h>
#include <limits.h>
#include <time.h>
int MatrixChainOrder(int p[], int n)
{
    int m[n][n];
    int i, j, k, L, q;
    // Cost is zero when multiplying one matrix
    for (i = 1; i < n; i++)
        m[i][i] = 0;
    // L is chain length
    for (L = 2; L < n; L++)
    {
        for (i = 1; i < n - L + 1; i++)
        {
            j = i + L - 1;
            m[i][j] = INT_MAX;
            for (k = i; k <= j - 1; k++)
```

```

        {
            q = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];
            if (q < m[i][j])
                m[i][j] = q;
        }
    }
}
return m[1][n - 1];
}
int main()
{
    int n, i;
    printf("Enter number of matrices: ");
    scanf("%d", &n);
    int p[n + 1];
    printf("Enter dimensions array (size %d):\n", n + 1);
    for (i = 0; i <= n; i++)
    {
        scanf("%d", &p[i]);
    }
    clock_t start, end;
    start = clock();
    printf("Minimum number of multiplications is %d\n",
        MatrixChainOrder(p, n + 1));
    end = clock();
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Execution time: %f seconds\n", time_taken);
    return 0;
}

```

Output:

```

C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 4\" && gcc 1.c -o 1 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 4\"1
Enter number of matrices: 4
Enter dimensions array (size 5):
3 4 5 2 3
Minimum number of multiplications is 82
Execution time: 0.001000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 4>

```

Q.N.2 Zero One Knapsack

Theory:

The 0/1 Knapsack problem is solved using dynamic programming where items are either taken completely or not taken at all. The goal is to maximize profit without exceeding the capacity.

Time Complexity: $O(nW)$

Space Complexity: $O(nW)$

Algorithm:

- a) Start
- b) Input number of items
- c) Input weight and profit of each item
- d) Input capacity of knapsack
- e) Create a DP table with rows as items and columns as capacity
- f) Initialize first row and column with zero
- g) For each item and capacity
- h) If item weight is less than capacity, choose maximum of including or excluding the item
- i) Otherwise exclude the item
- j) Store maximum value in the table
- k) The last cell contains maximum profit
- l) Print the maximum profit
- m) Stop

Code:

// Write a c program to implement for Zero One Knapsack.

```
#include <stdio.h>
#include <time.h>
// Function to return maximum value that can be put in knapsack
int max(int a, int b)
{
    return (a > b) ? a : b;
}
int knapsack(int W, int wt[], int val[], int n)
{
    int i, w;
    int K[n + 1][W + 1];
    // Build table K[][] in bottom up manner
    for (i = 0; i <= n; i++)
    {
        for (w = 0; w <= W; w++)
        {
            if (i == 0 || w == 0)
                K[i][w] = 0;
            else if (wt[i - 1] <= w)
                K[i][w] = max(val[i - 1] + K[i - 1][w - wt[i - 1]], K[i - 1][w]);
            else
                K[i][w] = K[i - 1][w];
        }
    }
    return K[n][W];
}
int main()
{
```

```

int n, W, i;
printf("Enter number of items: ");
scanf("%d", &n);
int val[n], wt[n];
printf("Enter values of items:\n");
for (i = 0; i < n; i++)
    scanf("%d", &val[i]);
printf("Enter weights of items:\n");
for (i = 0; i < n; i++)
    scanf("%d", &wt[i]);
printf("Enter capacity of knapsack: ");
scanf("%d", &W);
clock_t start, end;
start = clock();
printf("Maximum value in Knapsack = %d\n", knapsack(W, wt, val, n));
end = clock();
double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
printf("Execution time: %f seconds\n", time_taken);
return 0;
}

```

Output:

```

C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 4\" && gcc 2.c -o 2 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 4\"2
Enter number of items: 7
Enter values of items:
2 3 3 4 4 5 7
Enter weights of items:
3 5 7 4 3 9 2
Enter capacity of knapsack: 9
Maximum value in Knapsack = 15
Execution time: 0.000000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 4>

```

Q.N.3 Floyd Warshall

Theory:

Floyd algorithm is a dynamic programming method used to find the shortest path between all pairs of vertices in a weighted graph. It updates the shortest distance by considering intermediate vertices.

Time Complexity: $O(n^3)$

Space Complexity: $O(n^2)$

Algorithm:

- Start
- Input number of vertices
- Input adjacency matrix of the graph
- Initialize distance matrix with given weights

- e) For each intermediate vertex k
- f) For each pair of vertices (i,j)
- g) Update distance [i][j] if a shorter path is found through k
- h) Repeat until all vertices are considered
- i) Print shortest distance matrix
- j) Stop

Code:

// Write a c program to implement for Floyd Warshall Algorithms.

```
#include <stdio.h>
#include <time.h>
#define INF 999

void floydWarshall(int graph[10][10], int n)
{
    int i, j, k;
    int dist[10][10];
    // Initialize distance matrix
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            dist[i][j] = graph[i][j];
        }
    }
    // Floyd–Warshall algorithm
    for (k = 0; k < n; k++)
    {
        for (i = 0; i < n; i++)
        {
            for (j = 0; j < n; j++)
            {
                if (dist[i][k] + dist[k][j] < dist[i][j])
                {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
    // Print shortest distance matrix
    printf("\nShortest Distance Matrix:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            printf("%d\t", dist[i][j]);
```

```

        }
        printf("\n");
    }
}
int main()
{
    int n, i, j;
    int graph[10][10];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }
    clock_t start, end;
    start = clock();
    floydWarshall(graph, n);
    end = clock();
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Execution time: %f seconds\n", time_taken);
    return 0;
}

```

Output:

```

C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 4\" && gcc 3.c -o 3 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 4\"3
Enter number of vertices: 3
Enter adjacency matrix:
0 4 11
6 0 2
3 999 0

Shortest Distance Matrix:
0      4      6
5      0      2
3      7      0
Execution time: 0.002000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 4>

```

Conclusion:

In this lab, several problems were solved using the dynamic programming approach. The algorithms implemented included Matrix Multiplication, Matrix Chain Multiplication, 0/1 Knapsack, and the Floyd algorithm. Dynamic programming helps in solving complex problems by breaking them into smaller overlapping subproblems and storing their results to avoid repeated computations.

Lab 5: Backtracking

Q.N.1 N-Queen Problem

Theory:

The N Queen problem is a classic backtracking problem where N queens must be placed on an N×N chessboard so that no two queens attack each other. Backtracking tries possible positions and removes a queen if it leads to a conflict.

Time Complexity: $O(N!)$

Space Complexity: $O(N)$

Algorithm:

- a) Start
- b) Input the value of N for the chessboard size
- c) Place the first queen in the first row
- d) Check each column of the current row
- e) If placing a queen does not conflict with previously placed queens, place the queen
- f) Move to the next row and repeat the process
- g) If no valid position is found in a row, backtrack to the previous row and move the queen to the next column
- h) Continue this process until all queens are placed
- i) Print the valid arrangement of queens on the board
- j) Stop

Code:

// Write a c program to implement for N-Queen Problem.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 20

int board[MAX], n;
int isSafe(int row, int col)
{
    for (int i = 0; i < row; i++)
        if (board[i] == col || abs(board[i] - col) == abs(i - row))
            return 0;
    return 1;
}

void printSolution()
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
```

```

        printf("%s ", board[i] == j ? "1" : "0");
        printf("\n");
    }
    printf("\n");
}
void solveNQueens(int row)
{
    if (row == n)
    {
        printSolution();
        return;
    }
    for (int col = 0; col < n; col++)
        if (isSafe(row, col))
        {
            board[row] = col;
            solveNQueens(row + 1);
        }
}
int main()
{
    printf("Enter number of queens: ");
    scanf("%d", &n);
    clock_t start = clock();
    solveNQueens(0);
    clock_t end = clock();
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Execution time: %f seconds\n", time_taken);
    return 0;
}

```

Output:

```

C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 5\" && gcc 1.c -o 1 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 5\"1
Enter number of queens: 4
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0

Execution time: 0.000000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 5>

```

Q.N.2 Subset Construction Problem

Theory:

The subset construction problem generates all possible subsets of a given set using backtracking. The algorithm explores all possible combinations by including or excluding each element.

Time Complexity: $O(2^n)$

Space Complexity: $O(n)$

Algorithm:

- a) Start
- b) Input number of elements in the set
- c) Input the elements of the set
- d) Initialize a solution array to store subsets
- e) Start from the first element
- f) For each element decide whether to include it in the subset or exclude it
- g) Recursively repeat the process for the next element
- h) When all elements are processed, print the generated subset
- i) Backtrack to explore other possible subsets
- j) Continue until all subsets are generated
- k) Stop

Code:

// Write a c program to implement for Subset Construction Problem.

```
#include <stdio.h>
#include <time.h>
int set[20], subset[20];
int n;
void generateSubset(int i)
{
    if (i == n)
    {
        printf("{ ");
        for (int j = 0; j < n; j++)
        {
            if (subset[j] == 1)
                printf("%d ", set[j]);
        }
        printf("}\n");
        return;
    }
    subset[i] = 1;
    generateSubset(i + 1);
    subset[i] = 0;
```

```

        generateSubset(i + 1);
    }
int main()
{
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &set[i]);
    clock_t start = clock();
    printf("Subsets are:\n");
    generateSubset(0);
    clock_t end = clock();
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Execution time: %f seconds\n", time_taken);
    return 0;
}

```

Output:

```

C:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester>cd "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 5\" && gcc 2.c -o 2 && "c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 5\"2
Enter number of elements: 3
Enter elements:
1 2 3
Subsets are:
{ 1 2 3 }
{ 1 2 }
{ 1 3 }
{ 1 }
{ 2 3 }
{ 2 }
{ 3 }
{ }
Execution time: 0.003000 seconds

c:\Users\V I C T U S\OneDrive\Documents\Prajwal\5th Semester\Lab Report For DAA\Lab 5>

```

Conclusion:

In this lab, problems were solved using the backtracking technique. The algorithms implemented included the N Queen problem and the Subset Construction problem. Backtracking systematically explores all possible solutions by trying different choices and rejecting those that violate constraints.